

Grammarly, Under the Hood

A research-based map of what Grammarly and the newer Superhuman Go publicly disclose, how ambient writing help appears inside other apps, and which public projects are worth studying.

**THE HONEST
BOUNDARY**

This is not a leak or reverse-engineering claim. Superhuman published detailed engineering posts in September 2026, but production extension source, exact frontend implementation, full training pipeline, prompts, ranking logic, and internal datasets remain private. Each conclusion is labeled by evidence strength.

Executive takeaways

- Grammarly is still a writing product; Superhuman is the parent company name. Superhuman Go is the newer cross-app assistant layer, and its 2026 browser/desktop architecture is the strongest current public technical evidence. [S1, S4]
- The classic “injects itself” experience is event-driven: an extension matches a page, notices editable text, decorates errors in place, and opens suggestion UI on hover/click. Go adds a persistent panel, selected-text actions, chat, and agents configured to run on a schedule, event, or typing. [S4, S6, S8, S14]
- Current disclosed correction infrastructure blends fine-tuned LLaMA-family models, a model gateway/router, internal vLLM GPU serving and Databricks model serving. [S2, S3]
- GemType is the closest compact injected assistant; Harper is an offline deterministic engine; Thoth shows local-service composition but its inspected public snapshot has no root license file. [S9–S13]
- For Mama Budget, render an ordinary component in your own cart. Trigger on cart/checkout events, keep scoring deterministic, and use a model only for grandma’s wording.

Confidence key

Label	Meaning
Documented	Stated by Superhuman/Grammarly, Chrome documentation, or project source/docs. A project privacy claim remains the project author’s claim.
Observed in code	Confirmed in checked public repository by static inspection; projects were not installed or run.
Inference	Practical conclusion, not a claim about secret internals.
Not public	No reliable public disclosure found for the exact implementation detail.

What people see — and what triggers it

The Grammarly brand now sits inside Superhuman. Classic Grammarly focuses on the text a person is actively writing or editing; Go expands toward the contents of open apps/documents and, when connected, information from other work tools. That enables broader search/actions and makes context visibility and consent part of the product. Go’s 2026 architecture is not proof that every legacy Grammarly surface uses the same code. [S1, S4, S15]

Surface and presentation

Surface	What appears	How the user brings it forward
Browser writing field	Inline marks, nearby suggestion card, persistent Go panel.	Focus/type in an enabled editor; hover/click a mark; open Go or use a selected-text action. [S4]
Browser page context	Go can use open page/app context and selected material to answer or run an agent.	Open panel, ask, select text, or configure schedule, event, or typing triggers. App/data connectors widen scope. [S1, S4, S14, S15]
Go active signal	A notch at screen edge appears on hover or during an automatic task; a green dot / extension status indicates active work.	The user can tell when Go is active and turn it off per app/site. [S15]
Desktop apps	Go panel and writing tools alongside supported desktop contexts.	Launch/keep Go available; interact through app UI and platform adapter. Not every app/field is guaranteed. [S1]
Writing actions	Rewrite, adjust tone, shorten/expand, proofread, generate, or use agent workflows.	Select text or prompt the assistant; explicit action is the safest model for transformations. [S4]

Trigger chain

Step	Trigger / response	Practical implication
1 Match	Extension runs on pages/frames allowed by browser permissions. [S6]	Broad access makes “works anywhere” possible, but raises trust cost.
2 Detect	Find editable text and apply site exclusions. Go guide gives a 38 px editor-height threshold for initializing browser writing. [S4]	Trigger on a meaningful editor, not every DOM mutation.
3 Analyze	Client keeps context; completed sentence/punctuation or pause can trigger correction. [S2]	Debounce, keep UI responsive, discard stale responses.
4 Present	Inline marks stay near the problem; hover/click reveals fix; side panel handles broader work. [S4, S8]	Synchronize source text and accepted edits.
5 Agent	Agent invoked directly or configured to run on a schedule, event, or typing; actions pass through the app/action layer. [S1, S14]	Make proactive behavior legible and controllable.

**USEFUL
DISTINCTION**

The writing assistant is not “send every keystroke to an LLM.” Sources describe event-based checks, context synchronization, routing, and UI thresholds. Exact request cadence and per-site logic are not fully public.

Publicly disclosed architecture

The diagram combines two first-party views: Go’s cross-platform client architecture (2 Sep 2026) and Grammarly’s correction inference rollout (16 Sep 2026). It is a conceptual path, not a claim that every agent and suggestion follows one identical request. [S1, S2]

1 SURFACES Browser extension Desktop app + WebViews	2 CLIENT LAYER Platform adapters Text/context bridges	3 PRODUCT SERVICES Message bus + copy/context sync Grammar service + agents	4 MODEL ROUTER Gateway chooses route A/B, traffic split, fallback	5 INFERENCE Fine-tuned model Internal vLLM / managed serving
--	--	---	--	--

Schematic synthesized from published disclosures; Go and inference posts describe overlapping but distinct layers.

What the 2026 Go post discloses

- Superhuman chose Tauri for Go desktop after evaluating options including Electron. Web UI lives in WebViews with platform-specific native bridges; this does not mean the entire app is written in Rust. [S1]
- The browser extension supplies a platform adapter; desktop adapters expose host-specific capabilities. An app/action message bus lets agents be authored once; a shared context model makes relevant state available across connected surfaces. [S1]
- The interface combines a command panel, side panel, and agent view. It is one interaction model over different host apps, not one universal DOM overlay. [S1, S4]

Correction service path

For the classic red-underlined flow, Superhuman describes clients synchronizing writing context with a copy service; useful sentence completions call a grammar service, which routes requests to model serving. A gateway lets the company change serving providers without rewriting clients. [S2]

Layer	Evidence-backed role	Boundary
Client/extension	Monitors supported editors, syncs relevant writing state, renders UI. [S1, S2]	Production extension source/private browser APIs are not disclosed.
Grammar service/gateway	Receives correction requests, chooses model route, stages provider migration. [S2]	Internal topology, full request schema and ranking are private.
Model serving	Internal vLLM deployment plus managed serving; A/B and traffic shifting. [S2, S3]	Provider share is a dated snapshot, not a permanent setting.

Model method, serving stack, and scale

Correction-model evolution (Superhuman’s description)

Stage	Published method	Meaning
Early	Many manually written grammar rules. [S2]	Predictable for known cases; coverage/maintenance grow expensive.
Middle	Ensemble of smaller deep-learning models. [S2]	More context; routing and quality tradeoffs.
Recent	A fine-tuned LLaMA-family model >1B parameters prompted to output a corrected sentence. [S2]	Company reports improved quality; grammar-specific input, not ordinary chat.

This is the documented direction for red-underlined correction, not the full foundation model stack for every feature. Checkpoint, fine-tuning recipe, training corpus, evaluation set, inference code, and complete deployment configuration are not public. [S2]

Serving stack and rollout

Component	Reported detail	Why it matters
vLLM + Kubernetes / AWS EKS	Self-hosted GPU serving for internal models. [S2]	Control over versions, costs, data path; needs GPU operations.
Databricks Model Serving	Partner session describes Foundation Model APIs for red-underlines; cites an 80% rollout snapshot. [S3]	Managed capacity; 80% is that session’s state, not today’s permanent split.
FP8 + speculative decoding	Reported optimizations for throughput and speed. [S2, S3]	Can reduce latency/cost; validate quality and hardware.
Gateway + hybrid routing	Internal/external serving behind a route layer; traffic can shift and A/B test. [S2]	Provider flexibility with operational complexity.
Shadow / staged rollout	New route evaluated then traffic shifted gradually; 16 Sep post describes hybrid serving in production. [S2]	Compare quality/latency against baseline before cutover.

SOURCE
SNAPSHOTS

The partner session gives an 80% Databricks rollout for red-underlines. A later company post describes hybrid inference as the production architecture. These can be sequential snapshots; neither should be treated as an unchanging traffic percentage. [S2, S3]

Scale (company / partner claims)

Superhuman says its GEC service supports about 40 million daily active users and roughly 100 billion LLM requests per week. The Databricks session separately reports 200,000 requests per second as peak load for the ML production team’s LLM services, with GEC included; it reports an 80% Databricks rollout for red underlines and a 99.99% availability target. These are company/vendor claims, not per-user typing rates or proof that every keystroke invokes an LLM. [S2, S3]

How software “injects itself”

Three different patterns are often called injection: a browser extension on arbitrary sites; a native helper beside desktop apps; or an ordinary component inside a product’s own page. Their permission, compatibility, and trust costs differ.

SITE PAGE	CONTENT SCRIPT	MESSAGE	SERVICE WORKER	OVERLAY / PANEL
Editable field DOM + events	Matched page shared DOM access	Structured request extension API	Settings, routing, event tasks	Shadow-root card or side panel

Chrome isolates extension JavaScript from page JavaScript while allowing the content script to work with the shared DOM. [S6–S8]

Browser extension anatomy (Manifest V3)

- Manifest rules match page/frames. Content-script JavaScript runs in an isolated world, but can inspect or modify the shared DOM; isolation does not make page text nonsensitive. [S6]
- Message the extension worker/page rather than placing provider secrets or business logic in a webpage. The service worker is event-driven and can be suspended; persist state deliberately. [S7]
- Shadow DOM can shield overlay styles from page CSS, but does not solve text access, focus, selection, caret, scrolling, accessibility, or undo. GemType shows the UI pattern in code. [S9]
- Google Docs and custom canvas editors need specific bridges. Harper pairs a Google Docs MAIN-world bootstrap with its general extension content script. [S10]
- Broad `<all\_urls>` access enables broad coverage but gives broad access. Prefer narrow/optional host permission, explicit opt-out, and visible controls for text flow. [S6, S9–S11]

Privacy is part of the interface

Superhuman’s Go FAQ says Go works with material already open and visible on screen (with separate access for connected-app connectors), stores parts of prompts and on-screen context in chat history, and encrypts stored history in transit and at rest. It says sensitive information is automatically removed before text is processed, content is not used to train third-party models, and use for Superhuman’s own models depends on the user’s training preference. That is the current published policy, not proof of how every field behaves in every integration. [S5, S15]

Reliability traps

Failure	Why it breaks	Mitigation
Rerender	Saved DOM/range references go stale.	Re-resolve stable target + selection before applying.
Stale reply	Old model answer arrives after more typing.	Revision ID/hash; discard old responses.
CSS / z-index	Overlay inherits styles, vanishes, or overlaps.	Shadow root, positioning and scroll observers.
Sensitive fields	Password/payment/OTP content gets processed.	Exclusions, narrow grants, pause and opt-out.
Bad text edit	Wrong span changes; undo/caret breaks.	Minimal ranges, preserve selection, input events, undo.

Open projects worth studying

Repositories and docs were inspected statically, not installed or run. “Free to view” is not the same as licensed for reuse. Project privacy/quality statements remain claims until independently tested.

Project	What it is / stack	Reuse & privacy reality	Best use
GemType Apache-2.0	Closest direct analogue: MV3, content scripts, selection toolbar, Shadow DOM, Gemini JSON, service worker. v0.1.5; commit 29 Jul 2026. [S9]	BYOK; project policy says text goes directly to Google, not its server. API may incur cost/limits; broad page match. [S9]	Study injected editor UX and range handling.
Harper Apache-2.0	Rust lint engine <code>WASM`(` harper.js` )`); local/offline extension with background worker and Docs bridge. Commit 3 Oct 2026. [S10]</code>	Docs say English/offline; extension docs describe early stage; not a general rewrite agent. [S10]	Deterministic local grammar and editor adapters.
Toth License unclear	Chrome extension + local Python service, Ollama/LanguageTool, optional cloud models, native messaging, desktop apps. Commit 27 Aug 2026. [S11]	Policy says localhost by default and optional providers. No root LICENSE in checked snapshot; do not reuse code without permission. [S11]	Study local-server/native-host composition.
GECToR Apache-2.0	Grammarly historical model; token-level edit tagging, BERT/RoBERTa/XLNet/Transformers-era tooling. [S12]	2020 research artifact, not current production system or a cross-app assistant UI. [S12]	Compare rule/model edits; learn tag-based correction.
LanguageTool LGPL-2.1+	Java grammar/spelling engine, local server, many languages, integrations. [S13]	Engine != complete injected UI. Review LGPL and each integration separately. [S13]	Local grammar infrastructure / language breadth.

SHORTLIST

Inspect GemType for the injected overlay; Harper for a licensed offline engine; Toth for local service composition but inspect only until its licensing is clear. For your own cart, borrow the interaction and event timing—not the extension permission model.

What Mama Budget should borrow

Mama Budget controls its own cart. That removes the hardest Grammarly-style problem—arbitrary third-party DOMs. Use a normal app component subscribed to cart and checkout events, not a browser extension.

CART STATE	RULES	MAMA UI	USER ACTION	PAYMENT
Item, qty, context, currency	Need / want / depends + impact	Reason + meter + mood	Keep / edit / checkout	Demo ledger + praise

App-owned events and deterministic scoring; no arbitrary-site DOM access.

Suggested event contract

Event	Run	Response
item_added / quantity_changed	Recompute label, cart, category, projected budget; no model call for score.	Quiet need response; proportionate want reaction.
checkout_started	Evaluate whole cart vs remaining budget and house rules; freeze shown evidence.	Explain why the item crossed threshold; judge can continue/back.
payment_confirmed	Record demo/Nessie charge idempotently, then recompute ledger.	Meter falls; Proud mood when behavior merits it.
remittance_posted	Protected family support rule; update budget distinctly.	No scolding; optional home-currency context.

Recommended build shape

- Own-page UI: cart-row component plus persistent grandma panel. Use the existing app framework; no browser permissions, site matching, or injected CSS.
- Policy: pure, versioned function takes item/context/month budget/spent and returns label, protected flag, impact, explanation and rule ID. Use it in demo and scorecard.
- Persona: model receives approved persona + label + impact + short reason; it writes the line but cannot change classification. Add a deterministic fallback.
- Budget: Nessie is mock/demo data for this pitch. Seed a month; add the demo transaction idempotently; recompute from the ledger rather than incrementing UI counters.
- Validation: real mum/aunt labels are the human benchmark; simulated votes stay separate. Report agreement, need false-positive/scold rate, and merchant-category baseline separately.

HACKATHON CALL

This is not “Grammarly for shopping.” It borrows the right moment and calm inline interruption, then uses auditable spending rules where a general writing model would be unsafe. Your controlled cart makes the demo faster, clearer, and easier to judge.

Sources and reading route

Accessed / snapshot checked 3 October 2026. Sources are first-party product and engineering docs, official browser documentation, and project repositories.

S1 Superhuman Engineering — Building Go for Browser and Desktop (2 Sep 2026)

First-party architecture disclosure: browser extension, desktop wrapper, platform adapters, app/action message bus, context synchronization and shared agent UI.

S2 Superhuman Engineering — How We Scaled Our LLM Inference Infrastructure to Serve 100+ Billion Requests a Week (16 Sep 2026)

First-party description of correction-model serving, router/gateway, vLLM, Kubernetes/EKS, model migration and production rollout.

S3 Databricks / Superhuman — Running LLM inference at scale in production (2026 session)

Partner engineering session with model and serving details, including one rollout snapshot for red-underline traffic.

S4 Superhuman Help — Go browser extension user guide

Current user-facing browser workflow and supported ways to invoke Go.

S5 Superhuman Privacy Policy

Current company policy on content, screen context, AI processing, retention and controls; policy effective 6 Jul 2026.

S6 Chrome for Developers — Content scripts

Primary documentation for matching pages, isolated JavaScript worlds, DOM access and content-script permissions.

S7 Chrome for Developers — Messaging and extension service workers

Primary documentation for content-script/background messaging and the event-driven service-worker pattern.

S8 Chrome for Developers — Side Panel API

Primary documentation for extension-owned persistent side-panel UI.

S9 GemType repository and privacy policy

Static review of the public Manifest V3 extension, Shadow DOM overlay, structured Gemini requests, permissions and project privacy disclosure; repo snapshot v0.1.5, latest commit observed 29 Jul 2026.

S10 Harper repository and Chrome extension architecture

Static review of Apache-2.0 Rust/WASM engine and current extension source/docs; latest repository commit observed 3 Oct 2026. Docs describe the extension as early-stage.

S11 Thoth repository and privacy policy

Static review of extension, loopback service, native messaging, Ollama/LanguageTool claims and project privacy policy; latest commit observed 27 Aug 2026. No root license file was present in the inspected snapshot.

S12 Grammarly Research — GECToR repository

Public, historical Grammarly grammar-correction research implementation; useful for method study, not evidence of the current production model.

S13 LanguageTool repository

Open-source multilingual grammar/spelling engine and local-server reference; distinct from a complete injected assistant UI.

S14 Superhuman — Go 1.0: Custom, Proactive Agents Everywhere You Write (12 Aug 2026)

First-party product description of agent triggers (schedule, event, typing), Go app availability, cursor actions and Go online.

S15 Superhuman Help — Go Data & Privacy FAQ

Product-specific disclosure on visible screen context, connected apps, stored chat context, retention/deletion, provider training and Superhuman model-training controls.

How to read the evidence

Strongest: Superhuman's 2026 engineering posts describe its system; Chrome docs describe extension behavior; repositories describe the code at the checked snapshot. Weaker: project marketing about privacy/quality/free use; verify with network inspection and current provider terms before production. No private Grammarly implementation is inferred from a clone.